

2

Building a Multi-Agent System with MCP

In the previous chapter, we established the foundational skill of context engineering: transforming a vague prompt into a structured semantic blueprint. Now, we will apply that skill at a much larger scale to build a system of specialized AI agents that collaborate to solve complex problems using contexts to communicate. A single LLM is a brilliant generalist, but it's not an expert. For any complex, multi-step task, using a single LLM is inefficient and often fails. Therefore, we will architect a **Multi-Agent System (MAS)**, where the *context* we engineer expands beyond the content of a single message to define the very design of the system. This includes defining which agents are needed, their specific roles and capabilities, and the structured methods they use to communicate without losing information. This is the true scope of context engineering.

Further, to make our system of agents reliable, we will implement the **Model Context Protocol (MCP)**, a shared language that ensures our *engineered contexts* are passed between agents with perfect fidelity. In this hands-on section, we will build a complete MAS-MCP system from scratch. You will learn how to design the **Orchestrator** that manages the workflow and the **specialist agents** (Researcher and Writer) that execute the tasks. You will also see how error handling, message validation, and automated quality control are built directly into the workflow, ensuring the system can recover from failures and maintain factual accuracy. By the end, you will have a fully functional system that can take a high-level goal and autonomously manage a complex, multi-step process. In a nutshell, we will cover the following topics:

- Architecting a context-driven MAS workflow
- Building an MAS with MCP to manage contexts
- Error handling and validation
- The evolution of AI architecture
- Tools for building agent systems

Let's begin by architecting our system.

Architecting the MAS workflow with MCP

We need a clear architectural plan before writing even a single line of code. This section lays out that blueprint. We will deconstruct the overall goal into core components, define the role of each part of the system, and map the flow of communication that will bring our agent team to life.

First, let's define the two core concepts at the heart of this design, the **MAS** and **MCP**:

- **MAS**: We will design a system that can run multiple independent agents, each one specialized in a distinct task such as research, writing, or data analysis. By

giving each agent a clear context, we ensure it can excel at its specific responsibility.

- **MCP:** For our agents to collaborate, they need a shared language. MCP gives us the rules for how our agents pass tasks and information to one another. It provides a framework that ensures every message is structured, reliable, and perfectly understood.

Our MAS will consist of three distinct components. Each plays a specific role, and together they form a workflow capable of transforming a high-level user goal into a finished product:

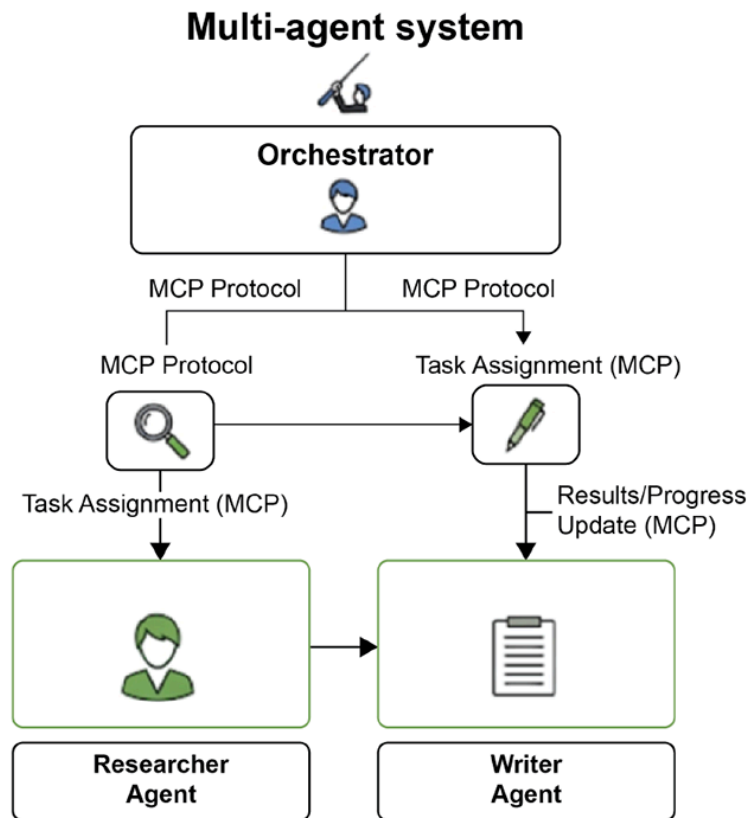


Figure 2.1: The architectural blueprint of our MAS workflow

The flowchart illustrates the system's complete workflow. Let's break down the role of each component in this cognitive pipeline:

- **Orchestrator (the project manager):** The Orchestrator is the brain of the operation. It doesn't perform specialized tasks itself but manages the entire workflow. It receives the user's high-level goal, breaks it down into logical steps, and delegates each step to the right agent. It is also responsible for receiving the results from one agent and passing them as *context* to the next. In other words, it applies *context chaining* at the system level to our MAS.
- **Researcher agent (the information specialist):** This is our first specialized agent. Its purpose is to take a specific topic, find relevant information, and synthesize that information into a structured summary. In our project, it will receive a research task from the Orchestrator and return the results as a clear, bullet-pointed list.
- **Writer agent (the content creator):** This is our second specialized agent. Its strength lies in communication and creative expression. It takes the structured summary from the Researcher and transforms it into a polished, human-readable piece of content, with careful attention to tone, style, and narrative.

Information doesn't just flow as raw text. Every interaction between agents is packaged as a structured **MCP message**. This ensures that tasks and results are always passed with *full context*, in a consistent, pre-

dictable, and reliable format. MCP is the connective tissue that turns a collection of individual agents into an integrated system.

With this blueprint in place, we're ready to begin building our MAS with MCP.

In this book, we explore the cutting-edge direction of applying MCP principles to agent-to-agent communication. While MCP was originally designed for agent-tool interaction, recent explorations (such as Microsoft's *Can You Build Agent2Agent Communication on MCP? Yes!*) demonstrate how MCP's evolving capabilities can support emerging inter-agent coordination patterns.

Link to the article: <https://developer.microsoft.com/blog/can-you-build-agent2agent-communication-on-mcp-yes>

Building an MAS with MCP

Now that we have a blueprint, it's time to start coding. In this section, we will implement the MAS and MCP step by step. We will begin with the system's core functionality, then return later to add error handling and validation in the *Error handling and validation* section.

Open `MAS_MCP.ipynb` from this chapter's repository to follow along. The initial OpenAI installation is the same as in `SRL.ipynb` from [Chapter 1](#).

We will work through the Colab notebook block by block: first defining our communication protocol, then building each specialist agent, constructing the Orchestrator to manage them, and finally, running the entire system to see our agent team in action. Let's first initialize the OpenAI client.

Initializing the client

We first initialize the OpenAI client, which will serve as our gateway to the LLM. We also import the `json` library to display our structured messages in a clean, readable format:

```
#@title 1. Initializing the Client
# -----
# We'll need the `openai` library to communicate with the LLM.
# Note: This notebook assumes you have already run a setup cell in your Colab
# environment to load your API key from Colab Secrets into an environment
# variable, as you specified.
# -----
import json

# --- Initialize the OpenAI Client ---
# The client will automatically read the OPENAI_API_KEY from your environment.
client = OpenAI()
print("OpenAI client initialized.")
```

The output displays a confirmation message:

```
OpenAI client initialized.
```

Defining the protocol

As we established earlier, our agents need a shared language to collaborate. That language is defined by MCP. In this section, we'll implement a simplified version to illustrate the process. While our Python dictionary approach works perfectly for learning, it's worth understanding the official MCP rules that govern how two systems communicate.

Message format

The structure of every MCP message is strictly defined to ensure consistency:

- All messages follow the **JSON-RPC 2.0** format as clean JSON objects
- Messages must be **UTF-8 encoded** for universal compatibility
- Each message must appear on a single line with no embedded newlines, making parsing fast and reliable

Once the message is defined, MCP defines the transport layers.

Transport layers

The transport layer defines how messages are transmitted between agents. The two primary methods are as follows:

- **STDIO (standard input/output)**: For agents running on the same machine (such as in our Colab notebook), they can communicate directly through standard input/output. This is the simplest and most direct method.
- **HTTP**: For agents running on different servers, messages are sent over the internet using standard HTTP requests.

Finally, we need a protocol management framework.

Protocol management

MCP also includes rules for compatibility and safety:

- **Versioning**: When using HTTP, a version header is required to ensure the client and server are using the same set of rules
- **Security**: There are rules for validating connections to prevent common cyberattacks and ensure you are communicating with the intended server

For the hands-on projects in this book, we will focus on the spirit of MCP by practicing structured communication. A simple Python dictionary is more than enough to illustrate this idea. It serves as our stand-in for the formal JSON-RPC object, letting us see clearly how messages are built and passed without getting lost in protocol overhead.

With that in place, we can now create a helper function called `create_mcp_message`. This function is our template for messages that travel through the system. By using the same structure each time, we make sure information never gets lost or misunderstood. Here's the code:

```
##@title 2. Defining the Protocol: The MCP Standard
# -----
# Before we build our agents, we must define the language they will speak.
# MCP provides a simple, structured way to pass context. For this example,
# our MCP message will be a Python dictionary with key fields.
# -----

def create_mcp_message(sender, content, metadata=None):
    """Creates a standardized MCP message."""
    return {
        "protocol_version": "1.0",
```

```
"sender": sender,
"content": content,
"metadata": metadata or {}
}
print("--- Example MCP Message (Our Simplified Version) ---")
example_mcp = create_mcp_message(
    sender="Orchestrator",
    content="Research the benefits of the Mediterranean diet.",
    metadata={"task_id": "T-123", "priority": "high"}
)
print(json.dumps(example_mcp, indent=2))
```

The output displays an example of our structured MCP message in JSON format:

```
--- Example MCP Message (Our Simplified Version) ---
{
  "protocol_version": "1.0",
  "sender": "Orchestrator",
  "content": "Research the benefits of the Mediterranean diet.",
  "metadata": {
    "task_id": "T-123",
    "priority": "high"
  }
}
```

The `create_mcp_message` function takes three inputs: the sender, the content (the task or information), and any optional metadata. It always returns a standard Python dictionary in the same structure. This consistency is the foundation of our system's reliability. Messages can flow between agents without risk of being lost, misread, or misunderstood.

With our protocol defined, we are ready to build the specialist agents.

Building the agents

This section contains the code that brings our agents from the workflow diagram in *Figure 2.1* to life. We will define each agent as a Python function. Every agent function will accept a structured MCP message as input and return another MCP message as output. An agent's specific role is shaped by its **system prompt**, which tells the LLM how to behave. To keep communication consistent, we will also create a single helper function called `call_llm` that manages all interactions with the OpenAI API.

Multi-Agent AI System Flowchart

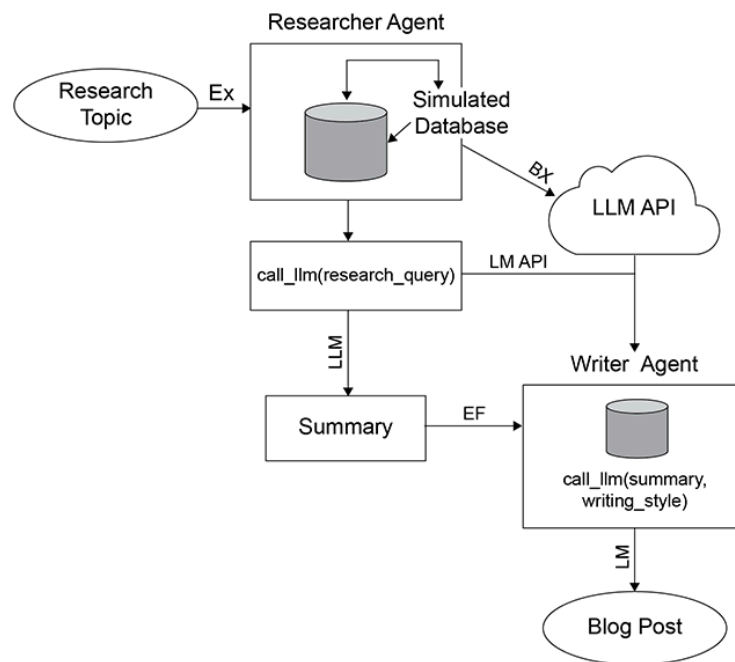


Figure 2.2: The architecture of the multi-agent functions

The figure illustrates a workflow that uses two AI agents to create a blog post. Information flows from an initial idea to a finished piece of content in a series of steps:

1. The process begins with a research topic, the prompt that starts the workflow. A message containing the topic is sent to our first agent, the **Researcher agent**, which gathers relevant facts and information from a simulated internal database.
2. Once the Researcher has the raw data, it uses the LLM API, an external language model service, to process the information and create a *summary* of the key findings. This summary is the bridge between the previous and next stages of the process.
3. The summary is then passed to the second agent, the **Writer agent**, whose role is to transform the research points into a finished article. The Writer also communicates with the LLM API, but for the complex and creative task of writing.
4. The final result of the system is a blog post, a clear demonstration of how specialized agents can work together to turn a high-level goal into a complete artifact.

We are now ready to implement the agents that communicate with one another.

Creating the helper function

Before we build the agents themselves, we need a reliable and consistent way for them to communicate with the LLM. Our helper function, `call_llm`, handles all API calls to OpenAI. It takes a **system prompt** and **user content** as input and returns the model's text response:

```

def call_llm(system_prompt, user_content):
    """A helper function to call the OpenAI API using the new client syntax."""
    try:
        # Using the updated client.chat.completions.create method
        response = client.chat.completions.create(
            model="gpt-5",
            messages=[

```

```

        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_content}
    ]
)
return response.choices[0].message.content...
```

This function has two inputs:

- `system_prompt`, which is a string that tells the model how to behave
- `user_content`, which is the string containing the specific information or question we are sending

Inside the function, we call the OpenAI client to create a chat completion. The messages are passed as a list of dictionaries, where we explicitly set the **system role** (to frame the agent's behavior) and the **user role** (to deliver the actual input). The function then returns the text from the model's response. The `try` and `except` block, meanwhile, ensures basic error handling, so the function doesn't crash if the API call fails.

With this helper function in place, we can now build our first specialized agent: the Researcher agent.

Defining the Researcher agent

Our first agent is the **Researcher**. Its job is to take a topic, look up relevant information, and summarize it for the next stage in the workflow.

We start by defining the function. The `researcher_agent` function takes in `mcp_input`, which follows the standard MCP message format we defined earlier. When activated, it prints a log message so we can trace when the agent starts working:

```

def researcher_agent(mcp_input):
    """
    This agent takes a research topic, finds information, and returns a summary.
    """
    print("\n[Researcher Agent Activated]")
```

Next, we create `simulated_database`, a simple dictionary that mimics a real data source. This is enough for our example at this point, though in later chapters, we will replace it with a vector database to support RAG:

```

simulated_database = {
    "mediterranean diet": "The Mediterranean diet is rich in fruits, vegetables, whole
}
```

From the input message, we extract the research topic. The agent reads the content field of the MCP message and looks up the corresponding entry in the database. If the topic isn't found, it returns a default message:

```

research_topic = mcp_input['content']
research_result = simulated_database.get(research_topic.lower(),
    "No information found on this topic.")
```

Now we define `system_prompt`. A prompt may seem simple, but within a MAS, it becomes a carefully engineered instruction. Here, the prompt

tells the LLM to behave like a research analyst and condense the retrieved information into 3–4 concise bullet points:

```
system_prompt = "You are a research analyst. Your task is to synthesize the provided i
```

The agent then calls our helper function, `call_llm`, passing in the system prompt and the research result. The summary generated by the LLM is stored in `summary`, and we print a log confirming that the work is complete:

```
summary = call_llm(system_prompt, research_result)
print(f"Research summary created for: '{research_topic}'")
```

Finally, the agent packages the summary into a new MCP message using `create_mcp_message`. This ensures the output is consistent with the protocol and includes both the content and relevant metadata:

```
return create_mcp_message(
    sender="ResearcherAgent",
    content=summary,
    metadata={"source": "Simulated Internal DB"}
)
```

The message output, complete with content and metadata, is the Researcher agent's final output. We can now build our Writer agent, which acts as the system's content creator and will transform the summary into a blog post.

Defining the Writer agent

The second agent in our system is the Writer. It receives the summarized research from the Researcher agent and transforms those summary points into a short blog post.

We begin by defining the function. The `writer_agent` function also takes an MCP message as input. The activation log helps us track when the Writer starts its work:

```
def writer_agent(mcp_input):
    """
    This agent takes research findings and writes a short blog post.
    """
    print("\n[Writer Agent Activated]")
```

Next, we extract the research summary from the message. This is the content produced earlier by the Researcher agent:

```
research_summary = mcp_input['content']
```

We then create `system_prompt` that sets the Writer's role. Unlike the Researcher, which focused on factual synthesis, the Writer is instructed to adopt an engaging, informative, and encouraging tone suitable for a health and wellness blog. The prompt also specifies the length (around 150 words) and asks for a catchy title:


```
system_prompt = "You are a skilled content writer for a health and wellness blog. Your
```

The agent calls `call_llm`, passing in the system prompt and the research summary. The returned text is our draft blog post. We also print a message to confirm that the draft was created:

```
blog_post = call_llm(system_prompt, research_summary)
print("Blog post drafted.")
```

Finally, the blog post is wrapped into a new MCP message using `create_mcp_message`. This ensures the output follows the same structured format as all other messages, with the content plus metadata; in this case, the word count of the draft:

```
return create_mcp_message(
    sender="WriterAgent",
    content=blog_post,
    metadata={"word_count": len(blog_post.split())}
)
```

This completes the Writer agent. It takes structured input from the Researcher, applies a new system prompt, and outputs a fully written blog post in a consistent MCP message format.

With both the Researcher and the Writer in place, we now have the core of our MAS defined. Each agent has a distinct role, and together they can carry out a task from raw information to finished content.

Building the Orchestrator

We now have our specialized agents, but we need a way to manage them. That role belongs to the **Orchestrator**. Think of it as the project manager of our AI team. Its job is to take a high-level goal, break it into a sequence of tasks, and delegate those tasks to the right agent. It also manages the flow of information, taking the output from one agent and passing it as input to the next. This creates a seamless process from start to finish:

The Orchestrator

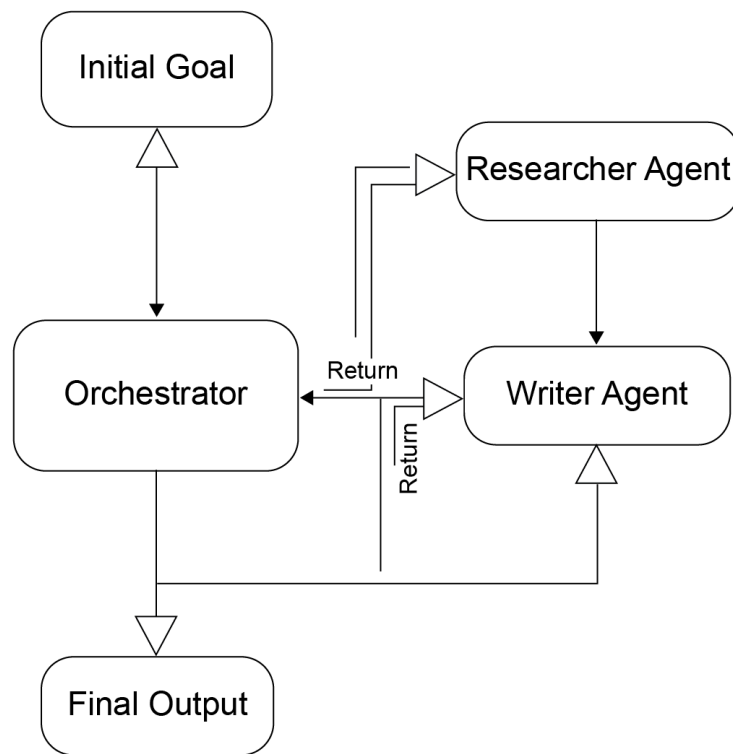


Figure 2.3: The Orchestrator of the agents

The workflow in the preceding figure begins when an **initial goal** is sent to the **Orchestrator**. The Orchestrator acts as a central hub, first sending a task to the **Researcher agent**. Once the research is complete, the agent sends its findings back to the Orchestrator. The Orchestrator then processes this information and sends a new task to the **Writer agent**. After the Writer finishes, it sends the completed content back to the Orchestrator. Finally, the Orchestrator assembles everything to produce the **final output**, completing the entire process.

The following code defines our Orchestrator. This single function manages the entire multi-agent workflow from start to finish. Its role is to call the Researcher, then the Writer, and finally, assemble the output into a completed artifact. Let's walk through it step by step:

```

def orchestrator(initial_goal):
    """
    Manages the multi-agent workflow to achieve a high-level goal.
    """
    print("=" * 50)
    print(f"[Orchestrator] Goal Received: '{initial_goal}'")
    print("=" * 50)
  
```

We begin by defining the `orchestrator` function. It accepts one argument, `initial_goal`, which represents the high-level task we want the system to complete. The function then prints confirmation that the goal has been received.

The first step is to delegate a research task. Here, we hardcode `Mediterranean Diet` as our topic, but in practice, this could be set dynamically based on the user's goal:

```
# --- Step 1: Orchestrator plans and calls the Researcher Agent ---
print("\n[Orchestrator] Task 1: Research. Delegating to Researcher Agent.")
research_topic = "Mediterranean Diet"
```

We wrap the research topic in an MCP message to keep communication consistent:

```
mcp_to_researcher = create_mcp_message(
    sender="Orchestrator",
    content=research_topic
)
```

The Orchestrator now calls the Researcher agent with the MCP message, receives the response, and prints the returned summary:

```
mcp_from_researcher = researcher_agent(mcp_to_researcher)
print("\n[Orchestrator] Research complete. Received summary:")
print("-" * 20)
print(mcp_from_researcher['content'])
print("-" * 20)
```

The Orchestrator now has the response from the Researcher agent in `mcp_from_researcher`, which it can pass on to the Writer agent. The Writer agent in turn first creates the MCP message in `mcp_to_writer` and sends it to the Writer agent, and then displays the result, which will appear when we run the whole system. We then log that the writing step is complete:

```
# --- Step 2: Orchestrator calls the Writer Agent ---
print("\n[Orchestrator] Task 2: Write Content. Delegating to Writer Agent.")
mcp_to_writer = create_mcp_message(
    sender="Orchestrator",
    content=mcp_from_researcher['content']
)
mcp_from_writer = writer_agent(mcp_to_writer)
print("\n[Orchestrator] Writing complete.")
```

Finally, the Orchestrator retrieves the Writer's response from the MCP message (`mcp_from_writer`) and presents the final result—the completed blog post:

```
# --- Step 3: Orchestrator presents the final result ---
final_output = mcp_from_writer['content']
print("\n" + "="*50)
print("[Orchestrator] Workflow Complete. Final Output:")
print("="*50)
print(final_output)
```

With this, our `orchestrator` function is complete. Let's now run the system and watch the workflow in action.

Running the system

We now have all the components in place: the Researcher agent, the Writer agent, and the Orchestrator. The final step is to run the system and

watch them work together.

This block of code defines a high-level user goal, passes it to the Orchestrator, and triggers the full workflow. With a single function call, the Orchestrator delegates to the Researcher, collects the summary, hands it off to the Writer, and assembles the final blog post:

```
##@title 5. Run the System
# -----
# Let's give our Orchestrator a high-level goal and watch the agent team work.
# -----
user_goal = "Create a blog post about the benefits of the Mediterranean diet."
orchestrator(user_goal)
```

Figure 2.4 visually maps the conversation between our agents. The Orchestrator begins by receiving the user's goal and delegating it to the Researcher agent, which generates a summary of key points. This summary is returned to the Orchestrator and passed to the Writer agent, which drafts the blog post and sends it back. Finally, the Orchestrator assembles and presents the completed blog, demonstrating how the agents work together to turn a single request into a finished output.

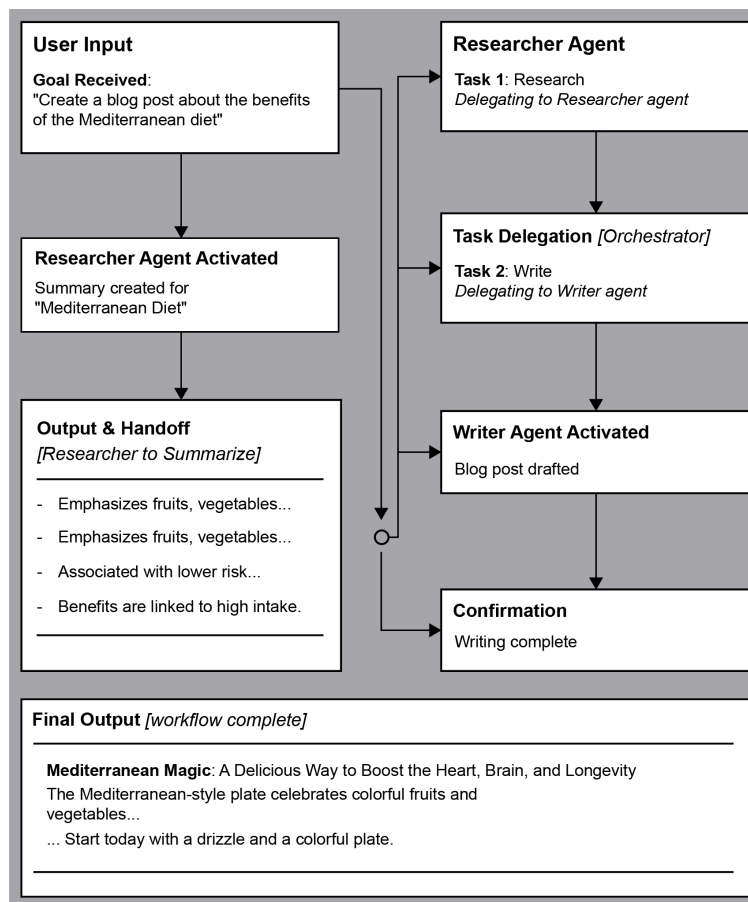


Figure 2.4: The flow of a context-driven MAS

Now, let's look at the actual output from our MAS. We'll break down the console log, showing how each agent contributes to the final blog post:

1. The system starts when we give the Orchestrator a goal. The Orchestrator acknowledges this goal and prepares to break it down into the first task:

```
=====
[Orchestrator] Goal Received: 'Create a blog post about the benefits of the Mediterranean diet.'
=====
Orchestrator Delegates to the Researcher
```

Next the Orchestrator knows it needs information. It announces the research task and activates the Researcher Agent. The Researcher then does its work and tells us it has created a summary:

```
[Orchestrator] Task 1: Research. Delegating to Researcher Agent.
[Researcher Agent Activated]
Research summary created for: 'Mediterranean Diet'
```

2. After the Researcher finishes its task, the Orchestrator confirms it has received the summary. It then displays the key points that the Researcher extracted. This output shows the core information that will be used for the blog post:

```
[Orchestrator] Research complete. Received summary:
-----
- Emphasizes fruits, vegetables, whole grains, olive oil, and fish as core foods.
- Associated with lower risk of heart disease, improved brain health, and increased longevity.
- Benefits are linked to high intake of monounsaturated fats (especially from olive oil) and antioxi
-----
```

3. With the summary in hand, the Orchestrator moves on to the next step. It announces that it is delegating the content writing task to the Writer agent. The Writer then begins to draft the blog post:

```
[Orchestrator] Task 2: Write Content. Delegating to Writer Agent.

[Writer Agent Activated]
Blog post drafted.
Writer Agent Completes the Draft
```

Once the Writer agent is done drafting the blog post, the Orchestrator confirms that the writing task is complete:

```
[Orchestrator] Writing complete.
```

4. Finally, the Orchestrator signals the completion of the entire workflow. It then presents the final blog post, which is the combined effort of all the agents working together:

```
=====
[Orchestrator] Workflow Complete. Final Output:
=====
Mediterranean Magic: A Delicious Way to Boost Heart, Brain, and Longevity

The Mediterranean-style plate celebrates colorful fruits and vegetables, hearty whole grains, silky
```

This run demonstrates the complete life cycle of our MAS. At this stage, the system works as intended. However, a network hiccup, a malformed message, or an unexpected LLM response could break the flow. To make the workflow more reliable and move closer to production-ready, we need to strengthen it with error handling, validation, and safeguards. That is exactly where we turn next.

Error handling and validation

The system we have built so far is a functional prototype; it works when everything goes according to plan. But in the real world, things rarely go

to plan. APIs can fail, messages can be malformed, and agents can return unexpected results. We must improve our system against these potential failures before we even consider it robust.

In this section, we will upgrade our simple script into a more resilient system. We'll add error handling to our API calls, create a validator to ensure MCP messages are well formed, and teach the Orchestrator how to keep the workflow running even when an agent fails or produces flawed output.

For this section, we'll be working in the `MAS_MCP_control.ipynb` notebook, which contains the full implementation. We'll focus on two core engineering principles, resilience and reliability:

- **Resilience:** Hardening the connection to the LLM so temporary issues (such as API timeouts or rate limits) don't crash the system
- **Reliability:** Ensuring message integrity so that agents always exchange predictable, valid MCP messages

Let's start by improving resilience. Our original `call_llm` function was intentionally simple. It made a single API request and returned the result. In production, that approach is too fragile. If the OpenAI API goes down, or if a request fails due to an unstable network, the entire workflow could collapse. To fix this, we'll build a more robust version of the function that can handle retries.

Building robust components for the LLM

The first step in strengthening our system is replacing the original `call_llm` with a more resilient function: `call_llm_robust`. This new version retries failed requests instead of giving up immediately, allowing the system to recover from temporary issues.

Before we define the function, we need the `time` library, which lets us pause execution between retries. This import is already included in the initialization section of `MAS_MCP_control.ipynb`:

```
import time
```

With that in place, here's the new function:

```
##@title 3. Building Robust Components
--- Hardening the call_llm Function ---
def call_llm_robust(system_prompt, user_content, retries=3, delay=5):
    """A more robust helper function to call the OpenAI API with retries."""
    for i in range(retries):
        try:
            response = client.chat.completions.create(
                model="gpt-4",
                messages=[
                    {"role": "system", "content": system_prompt},
                    {"role": "user", "content": user_content}
                ]
            )
            return response.choices[0].message.content
        except Exception as e:
            print(f"API call failed on attempt {i+1}/{retries}. Error: {e}")
            if i < retries - 1:
                print(f"Retrying in {delay} seconds...")
```

```

        time.sleep(delay)
    else:
        print("All retries failed.")
        return None

```

In this upgraded function, `call_llm_robust`, we wrap the API call in a `try/except` block inside a loop. If an error occurs, the system waits a few seconds before trying again, with `time.sleep` pausing execution between attempts. This retry mechanism makes the system more resilient to temporary network issues. If all attempts still fail, the function returns `None`, which signals to the Orchestrator that it must decide how to handle the failure.

With resilience in place for our API calls, let's now turn to reliability by introducing a function that validates every MCP message before it reaches an agent.

Validating MCP messages

For agents to communicate reliably, they must be able to trust the messages they receive. If a malformed message slips through, the entire workflow could fail. To prevent this, we introduce an **MCP validator**, a simple guardrail that checks that every message conforms to our protocol. The check happens before the message gets passed to an agent:

```

--- The MCP Validator ---
def validate_mcp_message(message):
    """A simple validator to check the structure of an MCP message."""
    required_keys = ["protocol_version", "sender", "content", "metadata"]

    if not isinstance(message, dict):
        print(f"MCP Validation Failed: Message is not a dictionary.")
        return False

    for key in required_keys:
        if key not in message:
            print(f"MCP Validation Failed: Missing key '{key}'")
            return False

    print(f"MCP message from {message['sender']} validated successfully.")
    return True

```

This function is a crucial guardrail. The Orchestrator will call `validate_mcp_message` before passing any message along, ensuring the structure is complete and predictable. It first checks that the input is a dictionary, then verifies that all required keys (`protocol_version`, `sender`, `content`, and `metadata`) are present. This prevents errors caused by malformed context slipping into the workflow.

With this validator in place, the Orchestrator can trust the messages it distributes, and the agents can focus on their tasks. Together with the `call_llm_robust` function, which adds resilience to API calls, we now have the foundation of a system that is both more reliable and more fault-tolerant.

Let's now add another safeguard by introducing controls to the specialist agents.

Adding agent specialization controls and validation

Our current agent team is functional, but it has a weakness: it trusts the Writer agent's output completely. Even with good context, LLMs can misinterpret facts or introduce hallucinations. To make our system more reliable, we need a quality control step that checks the Writer's work before it is accepted as final.

First, we update the existing `researcher_agent` and `writer_agent` in the `MAS_MCP_control.ipynb` notebook. We replace the old `call_llm` function with the new `call_llm_robust` function inside both agents:

```
#@title 4. Building the Agents: The Specialists
--- Agent 1: The Researcher ---
def researcher_agent(mcp_input):
    ... (code omitted for brevity) ...
    system_prompt = "You are a research analyst. Synthesize the provided information into
    # Now using the robust caller
    summary = call_llm_robust(system_prompt, research_result)
    ... (code omitted for brevity) ...

--- Agent 2: The Writer ---
def writer_agent(mcp_input):
    ... (code omitted for brevity) ...
    system_prompt = "You are a content writer. Take the following research points and writ
    # Now using the robust caller
    blog_post = call_llm_robust(system_prompt, research_summary)
    ... (code omitted for brevity) ...
```

This ensures that both the Researcher and the Writer can survive temporary API outages by retrying failed requests instead of failing outright. These agents are now more resilient, but we still need a way to verify that the Writer's draft matches the Researcher's findings. The next step is to expand our team from two to three specialists. We introduce the validator agent.

The validator agent's sole purpose is to act as a fact-checker. It compares the Writer's draft against the Researcher's summary. This comparison confirms factual consistency. It adds a vital quality control layer to our system:

```
# --- Agent 3: The Validator ---
def validator_agent(mcp_input):
    """This agent fact-checks a draft against a source summary."""
    print("\n[Validator Agent Activated]")

    # Extracting the two required pieces of information
    source_summary = mcp_input['content']['summary']
    draft_post = mcp_input['content']['draft']

    system_prompt = """
    You are a meticulous fact-checker. Determine if the 'DRAFT' is factually consistent wi
    - If all claims in the DRAFT are supported by the SOURCE, respond with only the word \
    - If the DRAFT contains any information not in the SOURCE, respond with \"fail\" and a
    """

    validation_context = f"SOURCE SUMMARY:\n{source_summary}\n\nDRAFT:\n{draft_post}"
    validation_result = call_llm_robust(system_prompt, validation_context)
```



```
print(f"Validation complete. Result: {validation_result}")

return create_mcp_message(
    sender="ValidatorAgent",
    content=validation_result
)
```

The validator agent has its own semantic blueprint. It requires two inputs: the source summary from the Researcher and the draft blog post from the Writer. The model returns either `pass` if the draft is consistent with the summary or `fail` with a short explanation if it is not.

This gives the Orchestrator a new safeguard: it no longer has to trust the Writer blindly. Instead, it can check whether the draft is factually aligned with the research before accepting it as final.

We will now add orchestration logic controls to bring the validator into the workflow.

The final Orchestrator with a validation loop

The original simple Orchestrator was a linear task manager. We replace it with a more intelligent `final_orchestrator`. This new version includes a validation and revision loop.

The workflow is no longer strictly sequential. After `writer_agent` produces a draft, the Orchestrator delegates to `validator_agent`. If the validation fails, the Orchestrator sends the draft back to the Writer and includes the validator's feedback. This creates a powerful self-correcting system that mimics a real-world editorial process.

We will now examine the key changes in the `final_orchestrator` function found in *Section 5* of the notebook.

Step 1 is similar to the original Orchestrator. The key addition is the immediate call to `validate_mcp_message`. If the Researcher returns a malformed message, or if the content is empty (which might happen if `call_llm_robust` returned `None`), the Orchestrator stops the workflow immediately:

```
##@title 5.The Final Orchestrator with Validation Loop
def final_orchestrator(initial_goal):
    ... (Initialization code omitted) ...

    # Step 1 Research and Immediate Validation
    # --- Step 1: Research ---
    print("\n[Orchestrator] Task 1: Research. Delegating to Researcher Agent.")
    # ... (Call to researcher_agent) ...
    mcp_from_researcher = researcher_agent(mcp_to_researcher)

    # New: Validate the message structure immediately
    if not validate_mcp_message(mcp_from_researcher) or not mcp_from_researcher['content']:
        print("Workflow failed due to invalid or empty message from Researcher.")
        return

    research_summary = mcp_from_researcher['content']
    print("\n[Orchestrator] Research complete.")
```

Steps 2 and 3 are then combined into an iterative loop. If validation *passes*, the draft is accepted and the loop ends. If validation *fails*, the Orchestrator sends the Writer's draft back along with feedback from the validator, asking for a revision. The loop allows a limited number of revisions (`max_revisions`) to prevent infinite cycles:

```
Step 2 and 3 The Iterative Loop
# --- Step 2 & 3: Iterative Writing and Validation Loop ---
final_output = "Could not produce a validated article."
max_revisions = 2
for i in range(max_revisions):
    print(f"\n[Orchestrator] Writing Attempt {i+1}/{max_revisions}")

    # Prepare context for the writer
    writer_context = research_summary
    if i > 0:
        # If this is a revision, add the validator's feedback
        writer_context += f"\n\nPlease revise the previous draft based on this feedback: {"

    # Call the Writer Agent and Validate
    mcp_to_writer = create_mcp_message(sender="Orchestrator", content=writer_context)
    mcp_from_writer = writer_agent(mcp_to_writer)

    if not validate_mcp_message(mcp_from_writer) or not mcp_from_writer['content']:
        print("Aborting revision loop due to invalid message from Writer.")
        break
    draft_post = mcp_from_writer['content']
```

After the Writer produces a draft, the Orchestrator immediately performs the new validation step and decision point. We package both `research_summary` and `draft_post` into a single MCP message for `validator_agent`:

```
# --- Validation Step ---
print("\n[Orchestrator] Draft received. Delegating to Validator Agent.")

# Prepare context for the Validator (needs both summary and draft)
validation_content = {"summary": research_summary, "draft": draft_post}
mcp_to_validator = create_mcp_message(sender="Orchestrator", content=validation_content)
mcp_from_validator = validator_agent(mcp_to_validator)

# Validate Validator output
if not validate_mcp_message(mcp_from_validator) or not mcp_from_validator['content']:
    print("Aborting revision loop due to invalid message from Validator.")
    break
validation_result = mcp_from_validator['content']

# Decision Point
if "pass" in validation_result.lower():
    print("\n[Orchestrator] Validation PASSED. Finalizing content.")
    final_output = draft_post
    break
else:
    print(f"\n[Orchestrator] Validation FAILED. Feedback: {validation_result}")
    if i < max_revisions - 1:
        print("Requesting revision.")
    else:
        print("Max revisions reached. Workflow failed.")
```

The workflow now includes the validator agent. After the Writer agent finishes, the Orchestrator sends the draft to the validator. A decision

point checks the result: if the validation fails, the Orchestrator initiates a feedback loop and sends the draft back to the Writer agent for revision. If the validation passes, the Orchestrator accepts the draft and proceeds to the final output.

This design makes the Orchestrator much more than a task dispatcher. It becomes the editor-in-chief, ensuring that outputs are both factually consistent and resilient to errors. With this validation loop in place, our workflow can now self-correct, producing more reliable results.

Let's now implement the final robust system.

Running the final robust system

This final code block runs the complete upgraded system. Instead of calling the simple prototype Orchestrator, it executes `final_orchestrator`. With this, we can observe the entire robust workflow in action, including the new validation and revision steps:

```
#@title 6.Run the Final, Robust System
user_goal = "Create a blog post about the benefits of the Mediterranean diet."
final_orchestrator(user_goal)
```

The output shows the enhanced logging. We can see the MCP message validations occurring after each agent finishes its task. We also see the full writing and validation loop in action:

```
[=====
[Orchestrator] Goal Received: 'Create a blog post about the benefits of the Mediterranean
=====

[Orchestrator] Task 1: Research. Delegating to Researcher Agent.

[Researcher Agent Activated]
Research summary created for: 'Mediterranean Diet'
MCP message from ResearcherAgent validated successfully.

[Orchestrator] Research complete.
...
What's more? Numerous studies indicate that adherence to the Mediterranean diet is your ti
```

With these upgrades, our MAS is now significantly more reliable. It can do the following:

- Handle API errors gracefully through retry logic
- Verify the structure of every MCP message to ensure consistency
- Apply an automated quality control process

It's clear now that AI systems are becoming increasingly performant and complex. To close this chapter, let's briefly explore the broader evolution of AI that we are witnessing today.

The evolution of AI architecture

Our approach to building AI is evolving. These models are powerful and know a lot, but they are not experts in any single area. A single model struggles to stay focused and accurate when the task changes.

We therefore moved from using one general model to creating a team of specialized agents. We used context engineering to define the agent's role and skills. Our Researcher and Writer agents are examples of this approach. Each one is designed to do its single job very well, as shown here:

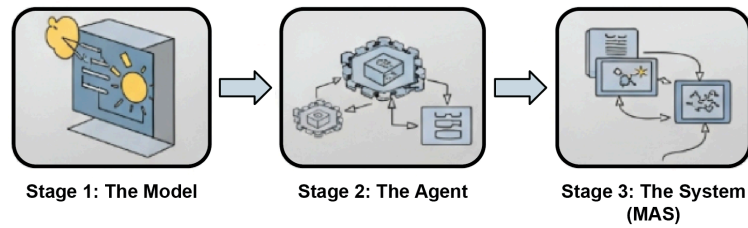


Figure 2.5: The evolution of AI architecture models to systems

A team of specialists needs coordination to solve big problems. The MAS provides the structure to manage these agents. MCP helps agents communicate reliably. Context engineering is the glue in this process: first used to improve responses from a single LLM, and now extended to define agents themselves and their interactions. These systems will expand and scale as AI is increasingly deployed in the real world.

Tools for building agent systems

So far, we've built an MAS from scratch. This hands-on approach gave us a strong foundation in the core ideas behind MAS and MCP: orchestration, structured communication, resilience, and validation.

In many projects, a customer or internal management may require that we avoid external platforms or frameworks, even open source ones that can be downloaded. Some may even insist on using only a local LLM. Fields such as aerospace, defense, banking, and finance often make this a strict prerequisite. In other cases, however, using a framework may be encouraged or even requested. So, what is the right path for a context engineer?

First, begin by understanding context engineering as we are doing here: from scratch. With that knowledge, you will be ready to implement a full custom solution or deploy an existing framework with an advantage over many other engineers. You will be both an architect and a problem-solver, finding ways to make the system work again when ready-to-use frameworks don't cut it. Here are some resources that you can explore:

- **The MCP specification:** The official MCP specification is a must-read. It details the formal rules for how agents should communicate. We built a simplified version. The official site defines the standards for JSON RPC transport layers and security: modelcontextprotocol.io.
- **MAS frameworks:** Several open source frameworks make it much easier to create MASs. They handle the hard parts, such as orchestration, communication, and agent management. These tools use concepts very similar to the ones we just built. Studying them is a great way to see how these ideas are used in production.
- **autogen:** autogen is a framework from Microsoft. It lets you build applications where multiple agents talk to each other to solve problems. autogen agents are flexible. They can combine LLMs human input and other tools to get work done. You can find autogen on GitHub here: github.com/microsoft/autogen.
- **CrewAI:** CrewAI is designed for building teams of role-based AI agents. It provides a structure for agents to work together on complex tasks. CrewAI is focused on collaboration and specialization. Its design is very similar to the

Orchestrator model we created. Find more information here: <https://www.crewai.com/>.

- **LangGraph:** LangGraph is part of the LangChain family of tools. It lets you build agent workflows as graphs. This is a powerful way to create systems that can iterate and correct themselves. LangGraph gives you very fine control over how information flows and how decisions are made. The LangGraph documentation can be found here: python.langchain.com/docs/langgraph.

With that, we have built an MAS and MCP use case from scratch to acquire a hands-on understanding of cutting-edge context-engineered systems. Let's summarize our journey.

Summary

In this chapter, we moved beyond single-agent interaction. We built a team of specialized agents, with each agent performing a specific function. We utilized MCP, which allows our agents to communicate using a standard structure. We then designed the system's architecture. We created a semantic blueprint to define the roles and workflow for our agent team. Next, we built the system one part at a time. We started with a function to create standard MCP messages. We then built each agent, giving each agent a unique skill with a specific system prompt. We built the Orchestrator to manage the context flow. It takes the output from the Researcher and passes it to the Writer. The final result was a functional MAS. It could take a user's goal and complete the research and writing process on its own. Finally, we made our prototype more robust by adding error handling and validation to the architecture.

You have now designed, implemented, and stabilized an MAS from scratch. You can use these advanced tools to build the next generation of AI solutions. Our next chapter will take us to the next level of our journey by opening our MAS to external data with RAG.

Questions

1. Is the main point of this chapter that one big LLM is best for complex jobs? (Yes or no)
2. Does the Orchestrator do the actual research and writing? (Yes or no)
3. Is MCP just a fancy way to make the LLM write better? (Yes or no)
4. Does the Researcher agent write the final blog post? (Yes or no)
5. Does the Writer agent get its task straight from the user's first request? (Yes or no)
6. Do you have to run each agent by hand to get the blog post? (Yes or no)
7. Does every MCP message have to include who sent it and what the content is? (Yes or no)
8. Is there just one big prompt that all the agents share? (Yes or no)
9. Do the agents in the notebook talk to each other over the internet using HTTP? (Yes or no)
10. Is the final product of this whole system just a research summary? (Yes or no)

References

1. Model Context Protocol. *Model Context Protocol*. <https://modelcontextprotocol.io>.
2. Weiss, G. (Ed.). (1999). *Multiagent Systems: A Modern Approach*. MIT Press.
3. Wooldridge, M. (2009). *An Introduction to Multiagent Systems*. John Wiley & Sons.
4. JSON-RPC Working Group. (2010). *JSON-RPC 2.0 Specification*. Accessed September 4, 2025. <https://www.jsonrpc.org/specification>.
5. Yao, S., et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv preprint arXiv:2210.03629.

6. Agent-to-agent MCP: <https://developer.microsoft.com/blog/can-you-build-agent2-agent-communication-on-mcp-yes>

Further reading

A Survey on Large Language Model based Autonomous Agents by L. Wang et al.: This paper provides a comprehensive overview of the entire agent landscape, defining the architecture and potential of modern AI agents.

<https://arxiv.org/abs/2308.11432>

Subscribe for a free eBook

New frameworks, evolving architectures, research drops, production breakdowns—*AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and GenAI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at <https://packt.link/80z6Y> or scan the QR code below.



Table of contents collapsed